

Wiki 1 – No Free Lunch (NFL) Theorem

1-Definition of NFL theorem

NFL theorem is used in machine learning.

It states that across all possible problems, no existing Algorithm is universally superior to another in performance; all optimization algorithms perform equally well.

2-What you understand by it (give examples, using algorithms you have used in your previous year's submissions/research)

A comparative study of machine learning models used in human activity recognition in wearable devices, conducted by Ciortuz et al.,(2025). Models ranging in complexity from traditional approaches like support vector machines (SVM) with hand created features and simple neural architectures such as convolutional neural networks (CNN), Gated Recurrent Units (GRUs), and Long Short-Term Memory Networks (LSTMs), to more sophisticated designs such as Transformer Networks and hybrid Convolutional LSTMs with and without attention mechanisms. The AF1 score results show nuances in performance of models under different factors such as class distribution, device positioning, and the nature of activities.

3-How it might help you in your broad understanding of computer science (particularly when you're trying to solve a problem you worked on in your previous years' work.

In a group work of benchmarking different models for human activity recognition, compared to group members' models, my LSTM approach outperformed Random Forest (62.33\%) due to its capability to model temporal patterns, while the CNN (86\%) excelled in spatial feature extraction but lacked sequential modeling. These comparisons underscored the LSTM's strengths in handling time-series data and revealed areas for further refinement, such as addressing data imbalance and exploring advanced architectures like Bidirectional LSTMs.

The NFL theorem implies that an algorithm that performs well on one dataset may perform poorly on another, even if both datasets relate to the same problem type.

This reinforces the need for experimentation and benchmarking, finding out the strengths and weaknesses of each optimiser in relation to the problem and dataset, and highlights that understanding the characteristics of the specific dataset is essential for choosing an effective algorithm for the problem.

Wiki 2 - Identify 2 problems:

mini project: utilizing Stochastic Diffusion Search (SDS) in skin cancer detection from images.

problem 1: Title: Low-Dimensional Parameter Optimisation for Skin Lesion Segmentation

Problem Description

In a full skin-cancer metastasis detection pipeline, segmentation is often needed to roughly isolate the lesion area from the surrounding skin. As a simplified and abstract version of this task, we consider a low-dimensional continuous optimisation problem in which a small number of segmentation-related parameters (e.g. two combined parameters representing colour and intensity thresholds) must be tuned.

Fitness function: Sphere function:

$$f(\mathbf{x}) = \sum_{i=1}^n x_i^2$$

for our problem:

$$f(\mathbf{x}) = x_1^2 + x_2^2.$$

where the decision vector is:

$$\mathbf{x} = (x_1, x_2) \in \mathbb{R}^2,$$

where each component represents an abstract segmentation parameter (e.g. combined threshold or scaling factors).

the objective is to minimise $f(\mathbf{x})$ subject to the constraints:

The global minimum is at $\mathbf{x}=(0,0)$ with $f(\mathbf{x})=0$

Solution encoding:

candidate solution:

$$\mathbf{x} = (x_1, x_2)$$

Each component x_i is stored as a floating-point number within a predefined range:

$$x_i \in [-5.0, 5.0], \quad i = 1, 2.$$

Estimation of the search space size:

In theory, for real-valued parameters the search space is uncountably infinite

$$\mathcal{X} = [-5, 5]^2 \subset \mathbb{R}^2.$$

For practical analysis, assume a discretisation. For example, suppose each parameter is discretised into $N=100$ equally spaced values in $[-5, 5]$. Then:

- Number of possible values per dimension: 100
- Number of possible combinations (search space size):

$$|\mathcal{X}_{\text{disc}}| = 100^2 = 10,000.$$

Constraints:

Bound constraints:

$$-5.0 \leq x_1 \leq 5.0, \quad -5.0 \leq x_2 \leq 5.0.$$

Evaluation constraints:

In a realistic setting inspired by metastasis detection (where each evaluation could correspond to running an image processing step), we assume a limited number of fitness evaluations, e.g.:

Maximum evaluations=5,000.

Problem: High-Dimensional Parameter Optimisation for a Simplified Metastasis Classification Pipeline

Problem description

Beyond segmentation, a metastasis detection system typically involves multiple stages (e.g. feature extraction, normalisation, and classification). Each stage has several tunable parameters. As a conceptual and simplified model of this multi-parameter tuning task, we consider a high-dimensional continuous optimisation problem.

each decision variable represents an abstract parameter (e.g. feature weights, normalisation factors, or classifier coefficients), but instead of using a real classifier

error function, we use a higher-dimensional Sphere function to study algorithm performance in a controlled setting.

Fitness function: Sphere function:

$$f(\mathbf{x}) = \sum_{i=1}^n x_i^2$$

the dimension of the problem is: $n=10$

let the decision vector be:

$$\mathbf{x} = (x_1, x_2, \dots, x_{10}) \in \mathbb{R}^{10}$$

representing 10 abstract parameters associated with a simplified metastasis classification pipeline.

The global minimum is at $\mathbf{x}=(0,0,\dots,0)$ with $f(\mathbf{x})=0$

Solution encoding:

candidate solution:

$$\mathbf{x} = (x_1, x_2, \dots, x_{10})$$

Each component x_i is stored as a floating point number within a predefined range:

$$x_i \in [-5.0, 5.0], \quad i = 1, \dots, 10.$$

Estimation of the search space size:

theoretical continuous search space:

$$\mathcal{X} = [-5, 5]^{10} \subset \mathbb{R}^{10},$$

which is unaccountably infinite.

for a discrete approximation:

Number of possible values per dimension: 100, number of possible combinations:

$$|\mathcal{X}_{\text{disc}}| = 100^{10}$$

since:

$$100^{10} = (10^2)^{10} = 10^{20}$$

Constraints:

Bound constraints:

$$-5.0 \leq x_i \leq 5.0, \quad i = 1, \dots, 10.$$

Evaluation constraints:

given that in the real metastasis classification task, evaluating a parameter set may be expensive (training a classifier, computing features on large image sets, etc.), we impose a limit on the number of fitness evaluations, for example: maximum evaluations = 20,000

References:

Fornasier, M., Huang, H., Pareschi, L. and Sunnen, P., 2022. Anisotropic diffusion in consensus-based optimization on the sphere. *SIAM Journal on Optimization*, 32(3), pp.1984-2012. Li, J., Cheng, K., Wang, S., Morstatter, F., Trevino, R.P., Tang, J. and Liu, H., 2017. Feature selection: A data perspective. *ACM computing surveys (CSUR)*, 50(6), pp.1-45. Supriyanto, C., Salam, A., Zeniarja, J., Utomo, D.W., Dewi, I.N., Paramita, C., Wijaya, A. and Safar, N.Z.M., 2025. A Bibliometric Review of Deep Learning Approaches in Skin Cancer Research. *Computation*, 13(3), p.78.

Wiki 3 – DFO

Results of running DFO 30 times on Sphere function:

	Mean	Median	Min	Max	Standard deviation
Sphere function	2.49e-19	4.98e-20	1.26e-21	3.57e-18	6.48e-19
Rastrigin function	1.30e-07	3.41e-13	1.14e-13	3.90 e-06	6.99e-07

Observations:

The table results indicate that the algorithm's performance on the sphere function is extremely close to the global minimum across all 30 runs (fitness values are practically 0).

DFO reliably converges to optimum in the setting of $1e-18$ – $1e-21$ → low absolute spread.

The few differences are at a numeric noise level (floating-point precision).

The algorithm's performance on the rastrigin function:

Many runs found near global minima (median and min values are essentially zero ($3.4e-13$ and $1.1e-13$))

Mean is much larger ($1.3e-07$) and StdDev is larger still ($6.99e-07$), and Max = $3.9e-06$.

Distribution is skewed as most runs hit very low fitness, but a few runs produce noticeably worse final fitness and inflate the mean and the standard deviation. The median shows higher than 50% success, while mean and standard deviation indicate occasional outliers.

Instability shows in some seeds by rastrigin worst fitness being small in absolute terms but larger than the best runs in absolute terms.

Possible causes can represent in:

Sphere being easy where gradient-free local moves quickly converge because there are no local traps,

Rastrigin is multimodal: DFO can find global basin in many runs but occasionally gets stuck in local minima which explains skew and higher variance.

Disturbance threshold ($\delta = 0.001$) is very small: It gives very rare random re-initialization per dimension; good for exploitation but weak for escaping local minima.

On Rastrigin, the small delta may be enough for many runs (combined with neighbor-based moves) but insufficient for some runs → those outliers.

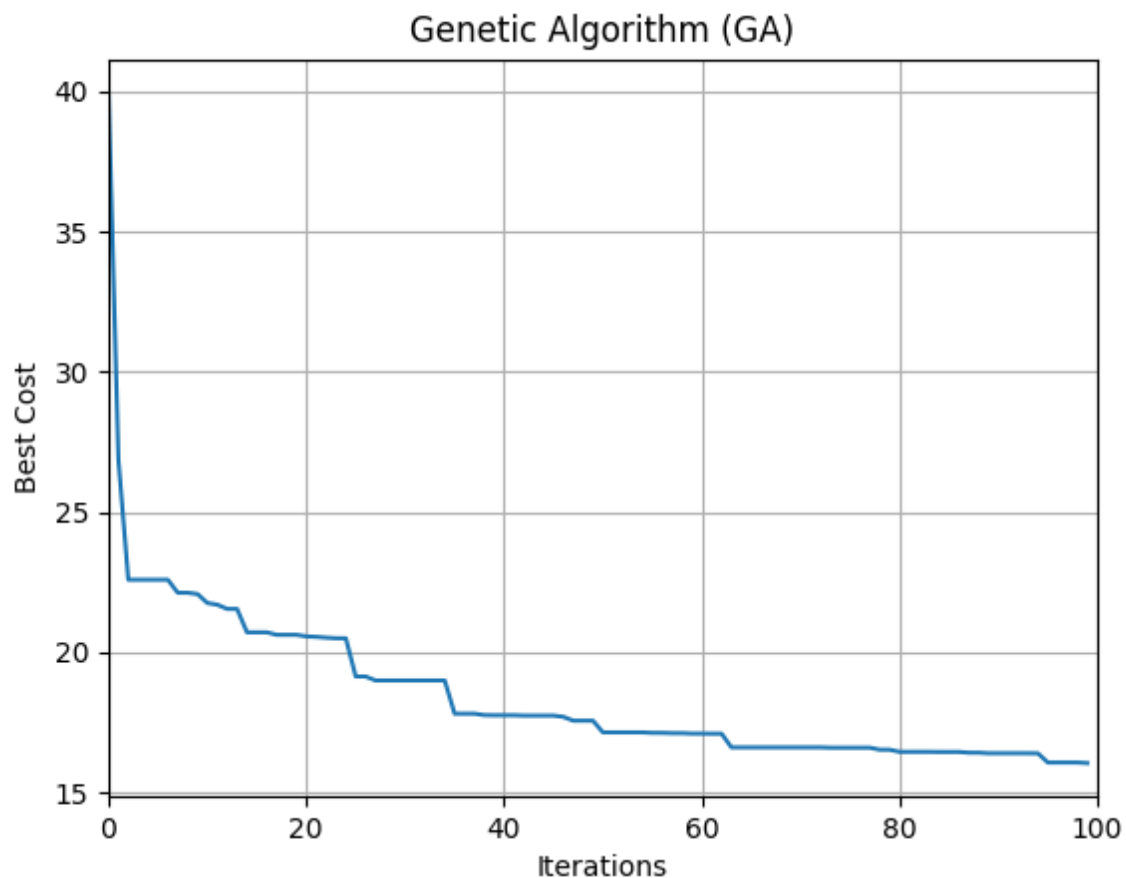
The process required to achieve the goal of finding the bright object/s in the input image using DFO:

The image is converted to a single brightness channel, then an optional small Gaussian blur is applied to reduce pixel noise. A fitness function (eg. Sphere function) that returns brightness at a fly's continuous (x,y) coordinate (using bilinear interpolation) is defined, then you decide whether to minimize negative brightness or maximize brightness directly. A population of N flies with random continuous coordinates inside the image bounds is initialized, then the DFO main loop is started. Each iteration every fly's fitness is evaluated, then the global best fly is identified. For each non-best fly its left and right neighbours are compared and the better neighbour bNeighbour is chosen, then for each coordinate d with probability delta that coordinate is reinitialized and otherwise

updated using the update function: $X[i,d] = X[bNeighbour,d] + u \cdot (X[s,d] - X[i,d])$ (with u sampled in $[0,1]$). Out-of-bounds coordinates are clamped or reinitialized, then the next iteration begins until max iterations or a stopping condition is reached. After termination final fly positions are collected, then clustering (e.g., DBSCAN or distance thresholding) is applied to group flies into candidate objects. Cluster centers are computed and reported as bright object locations, then optional local refinement or intensity checks are performed to verify and rank detections. For robustness, increase N or iterations to improve coverage, adjust delta higher early and lower later to balance exploration and exploitation, and avoid plotting every iteration to keep the run fast.

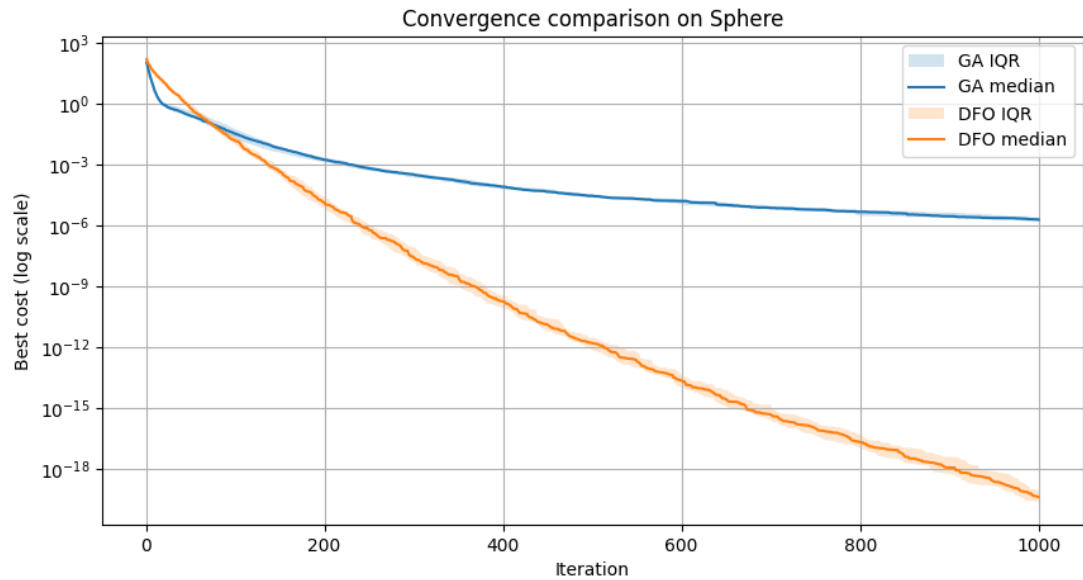
Wiki 4 – Genetic Algorithm

1-

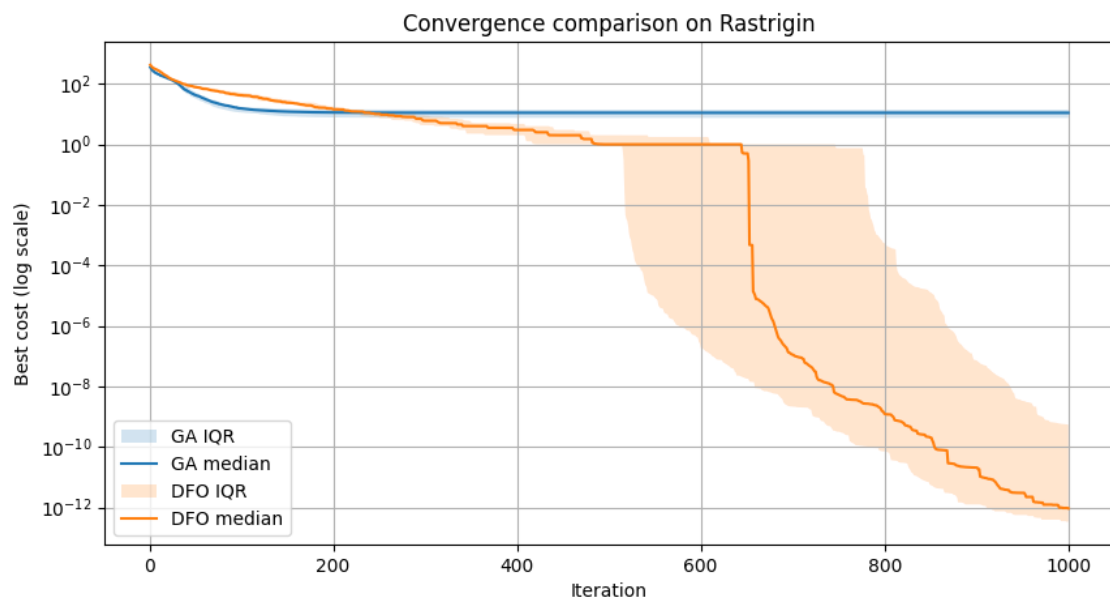


2- the fitness function used in the code is the sphere function.

3-Comparison between GA and DFO when run for 1000 iterations



On the Sphere problem, DFO consistently outperforms GA, its median falls steadily to numerical zero with a very tight IQR, whereas the GA improves more slowly and remains orders of magnitude worse by the end.



On the Rastrigin problem, both algorithms start similarly but the GA's median quickly plateaus while DFO begins a strong downhill phase after roughly 400–600 iterations, ultimately reaching much lower costs by iteration 1000 (though with a wide IQR indicating run-to-run variability and some runs lagging).

Comparison/ conclusion:

GA gives faster early exploration but tends to get stuck on hard multimodal landscapes, while DFO provides stronger steady exploitation and can find much deeper minima given enough iterations (at the cost of higher variability on difficult problems).

Challenge 1-

	Brighton	Bristol	Cambridge	Glasgow	Liverpool	London	Manchester	Oxford
0 Brighton	0	172	145	607	329	72	312	120
1 Bristol		0	192	494	209	158	216	92
2 Cambridge			0	490	237	75	205	100
3 Glasgow				0	286	545	296	489
4 Liverpool					0	421	49	208
5 London						0	249	75
6 Manchester							0	194
7 Oxford								0

Given the distances provided above (distances for private jet in kilometres) to find the best solution (i.e. shortest distance) for visiting each city once.

The shortest distance is: 1355 (round trip/coming back to the initial city)

The order to visit the cities (round trip):

Brighton -> Oxford -> Bristol -> Liverpool -> Glasgow -> Manchester -> Cambridge -> London -> Brighton

Card Game:

Search Space size: $2^{10} = 1024$

Potential Solution encoding:

- Represent each candidate partition as a fixed-length 10-bit vector (array or list of 0/1). Bit i (0-based) corresponds to card value $i+1$. A bit value of 1 means card $(i+1)$ is placed in Group A (the SUM group, target = 36). A bit value of 0 means the card goes to Group B (the PRODUCT group, target = 360). This bitstring is compact, natural for crossover/mutation, and encodes every possible labeled split (1024 possibilities).

Getting sum and product from a genotype:

- From the genotype, build Group A by collecting all cards whose bit = 1 and Group B by collecting those with bit = 0.
- Compute $\text{sumA} = \text{sum}(\text{Group A})$.

- Compute $\text{prodB} = \text{product}(\text{Group B})$. If Group B is empty treat $\text{prodB} = 1$ (neutral product). These are the raw objective values you want to match to the targets 36 and 360.

Solution evaluation:

Using the sphere fitness function:

Mapping the partition to a numeric vector for Sphere

- To use the Sphere function, the partition error must be converted into a small continuous vector $x = (x_1, x_2)$.
- Choose a mapping that makes the two components comparable:

$x_1 = (\text{sumA} - 36) / \text{scale_sum}$. scale_sum is a positive constant (suggested default = 10) used to normalize typical sum deviations so x_1 is $O(1)$.

$x_2 = \log(\text{prodB} + \text{eps}) - \log(360)$. Using the logarithm compresses the multiplicative range of products and makes matching a ratio into an additive difference; eps (e.g. $1e-9$) avoids $\log(0)$.

note: sum errors are additive differences while product errors are multiplicative; the log for product and a scaling of sum make the two error axes comparable for the Sphere metric.

Compute $\text{Sphere}(x) = x_1^2 + x_2^2$. This is the minimized value: the closer sumA is to 36 and prodB is to 360 (in log-space), the closer x will be to $(0,0)$ and the smaller the Sphere value.

Interpretation: Sphere gives an L2 penalty on the normalized sum error and the log-product error. It is smooth, convex in the error coordinates, and drives both objectives toward their targets.

Run GA in two modalities: Generational GA and (more iterations using)

Steady State GA. Repeat the experiments 50 times in each mode.

Variance report the results:

	mean	median	var	Std	min	max
Generational GA	0.005	0.005	2.16	0.005	7.71	0.021
Steady-State GA	0.007	0.005	5.69	0.008	7.71	0.042

evaluation of the results:

Both GAs reached very small Sphere fitness values (means ≈ 0.005 and 0.007 ; medians equal at 0.005), so typically both methods produce near-optimal partitions.

Generational GA has a slightly better mean and noticeably lower variance/std — i.e., it is a bit more consistent across runs than the Steady-State GA.

Consistency and spread

- Generational GA: mean = 0.005 , median = 0.005 , std ≈ 0.005 → distribution is tight and centered near 0.005 .
- Steady-State GA: mean = 0.007 , median = 0.005 , std ≈ 0.008 → median equals Generational but the larger mean and std indicate more outliers / worse runs pull the average up.
- The higher variance (given for Steady-State as 5.69) confirms higher run-to-run variability than Generational (2.16). In plain terms: Steady-State sometimes finds very good solutions but also has more runs that do worse.

Conclusion:

Generational GA is a bit more reliable (lower average error and lower spread).

Steady-State GA can reach similar medians but is less stable and produces more bad outliers.

Wiki 5 – Stochastic Diffusion Search

Unlike traditional optimisation methods that rely on exhaustive evaluation, SDS uses partial evaluations and information diffusion to locate high-quality solutions efficiently.

The ultimate simple way of explaining Stochastic Diffusion is through the mining game metaphor:

- A group of miners (agents) search for gold in a large mountain range (search space).
- The mountains are divided into hills (possible solutions).
- Each miner randomly selects one hill, called their hypothesis.
- Each day, a miner only tests one randomly chosen seam on their hill.

Test Phase

At the end of each day:

- If a miner finds enough gold, they become happy (active).
- If not, they become unhappy (inactive).

This represents partial hypothesis evaluation, meaning that only a small portion of the full solution is tested.

Diffusion Phase

During the diffusion stage:

- Each unhappy miner randomly selects another miner.
- If the selected miner is happy, the unhappy miner adopts their hypothesis.
- If the selected miner is also unhappy, the miner chooses a new hypothesis at random.

Over repeated iterations, the population of miners naturally concentrates around the hill with the highest gold density, demonstrating how SDS achieves self-organisation and convergence without central control.

If your search space is a large image, and your model is a small image

The micro features can be:

Search space (SS) → Large image

Model → Small template image

Hypothesis → A possible location of the small image inside the large image

Micro-feature → A single pixel or small pixel region chosen from the hypothesised location.

Following the SDS rule used in the text example (SDS lecture slides)

Instead of comparing the entire small image, each agent would:

- Randomly select one pixel (or very small patch)
- Compare it to the corresponding pixel in the model
- If the values match well, the agent becomes active
- The correct image location spreads through diffusion.

Other possible micro-features:

- A single grayscale intensity value
- A single RGB colour value
- A small 2×2-pixel patch

References:

COMP-1805 Lecture slides

Al-Rifaie, M.M., 2014. Dispersive Flies Optimisation. In Proceedings of the 2014 Federated Conference on Computer Science and Information Systems (FedCSIS), vol. 2 (ACSIS), pp. 529–538.

Bishop, J.M., 1989. Stochastic Searching Networks. In Proceedings of the 1st IEE International Conference on Artificial Neural Networks, London, UK, pp. 329–331.

Ciortuz, G., Pour, H.H. and Fudickar, S.J.F., 2025. Machine learning models for wearable-based human activity recognition: A comparative study. *Neurocomputing*, (Article 130911). (in press).

Fornasier, M., Huang, H., Pareschi, L. and Sunnen, P., 2022. Anisotropic diffusion in consensus-based optimization on the sphere. *SIAM Journal on Optimization*, 32(3), pp. 1984–2012.

GeeksforGeeks, 2024. Genetic Algorithms. [Online]. Available at: <https://www.geeksforgeeks.org/dsa/genetic-algorithms/> (Accessed 8 December 2025).

Li, J., Cheng, K., Wang, S., Morstatter, F., Trevino, R.P., Tang, J. and Liu, H., 2017. Feature selection: A data perspective. *ACM Computing Surveys*, 50(6), Article 94, pp. 1–45.

Supriyanto, C., Salam, A., Zeniarja, J., Utomo, D.W., Dewi, I.N., Paramita, C. and Safar, N.Z.M., 2025. A bibliometric review of deep learning approaches in skin cancer research. *Computation*, 13(3), p. 78.

Wolpert, D.H. and Macready, W.G., 1997. No free lunch theorems for optimization. *IEEE Transactions on Evolutionary Computation*, 1(1), pp. 67–82.

Declaration of AI Use

I have used AI while undertaking my assignment in the following ways:

- To develop research questions on the topic – NO
- To create an outline of the topic – NO
- To explain concepts – YES
- To support my use of language – YES/
- In other ways, as described below:
to explain Algorithm concepts and to guide with potential future improvement of algorithms' performance.

Appendix:

DFO python code:

```
import numpy as np
import time
```

```
# FITNESS FUNCTION (SPHERE FUNCTION)
```

```
def f(x):
```

```
    """Sphere function: sum of squares. x is a 1-D array-like."""
```

```
    x = np.asarray(x, dtype=float)
```

```
    return float(np.sum(x * x))
```

```
def run_dfo(
```

```
    N=100,
```

```
    D=30,
```

```
    delta=0.001,
```

```
    maxIterations=1000,
```

```
    lower=-5.12,
```

```
    upper=5.12,
```

```
    seed=None,
```

```
    verbose=False,
```

```
):
```

```
    """
```

```
    Run a single instance of the DFO algorithm and return the best fitness found and best solution.
```

```
    If `seed` is not None, np.random.seed(seed) will be set for reproducibility.
```

```
    """
```

```
    if seed is not None:
```

```
        np.random.seed(seed)
```

```

lowerB = [lower] * D
upperB = [upper] * D

# INITIALISATION PHASE
X = np.empty((N, D), dtype=float) # population matrix
fitness = [None] * N

# INITIALISE FLIES WITHIN BOUNDS (vectorized)
for i in range(N):
    X[i, :] = np.random.uniform(lower, upper, size=D)

# MAIN DFO LOOP
for itr in range(maxIterations):
    # EVALUATION
    for i in range(N):
        fitness[i] = f(X[i, :])
    s = int(np.argmin(fitness)) # index of best fly

    if verbose and (itr % 100 == 0):
        print(
            f"Iteration: {itr}\tBest fly index: {s}\tFitness value: {fitness[s]:.6e}"
        )

    # UPDATE EACH FLY
    for i in range(N):
        if i == s:
            continue # elitist strategy: skip the best fly

        left = (i - 1) % N
        right = (i + 1) % N
        bNeighbour = right if fitness[right] < fitness[left] else left

        for d in range(D):
            if np.random.rand() < delta:
                X[i, d] = np.random.uniform(lowerB[d], upperB[d])
                continue

            u = np.random.rand()
            X[i, d] = X[bNeighbour, d] + u * (X[s, d] - X[i, d])

```

```

# OUT OF BOUND CONTROL
if X[i, d] < lowerB[d] or X[i, d] > upperB[d]:
    X[i, d] = np.random.uniform(lowerB[d], upperB[d])

# final evaluation
for i in range(N):
    fitness[i] = f(X[i, :])
s = int(np.argmin(fitness))
return float(fitness[s]), X[s, :].copy()

def main():
    runs = 30
    best_values = np.empty(runs, dtype=float)
    seeds = list(range(runs)) # reproducible: use seeds 0..29

    t0 = time.time()
    for run_idx in range(runs):
        seed = seeds[run_idx]
        best_fit, best_sol = run_dfo(seed=seed, verbose=False)
        best_values[run_idx] = best_fit
        # optional: print per-run summary
        print(f"Run {run_idx+1:02d}/{runs} (seed={seed}) -> best fitness: {best_fit:.6e}")
    t1 = time.time()

    # SUMMARY STATISTICS
    mean_val = np.mean(best_values)
    median_val = np.median(best_values)
    min_val = np.min(best_values)
    max_val = np.max(best_values)
    std_val = np.std(best_values, ddof=0)

    print("\nSummary over {} runs:".format(runs))
    print(f"Mean   : {mean_val:.6e}")
    print(f"Median : {median_val:.6e}")
    print(f"Min    : {min_val:.6e}")
    print(f"Max    : {max_val:.6e}")
    print(f"StdDev : {std_val:.6e}")
    print(f"\nElapsed time: {t1 - t0:.2f} seconds")

    # If you also want the full vector of all run results, it's in `best_values`.

```

```
return best_values
```

```
if __name__ == "__main__":  
    main()
```

Rastrigin Function Code:

```
# FITNESS FUNCTION (RASTRIGIN FUNCTION)
```

```
def f(x, A=10.0):
```

```
    """
```

```
    Rastrigin function:
```

```
     $f(x) = A * n + \sum_{i=1..n} (x_i^2 - A * \cos(2 * \pi * x_i))$ 
```

```
    Global minimum at x = 0 with f(x) = 0.
```

```
    """
```

```
    x = np.asarray(x, dtype=float)
```

```
    n = x.size
```

```
    return float(A * n + np.sum(x * x - A * np.cos(2.0 * np.pi * x)))
```

```
def run_dfo(
```

```
    N=100,
```

```
    D=30,
```

```
    delta=0.001,
```

```
    maxIterations=1000,
```

```
    lower=-5.12,
```

```
    upper=5.12,
```

```
    seed=None,
```

```
    verbose=False,
```

```
):
```

GA modalities card game code

```
import numpy as np
```

```
import math
```

```
import csv
```

```
from typing import List, Tuple
```

```

# ----- Problem & encoding -----
CARDS = list(range(1, 11))
TARGET_SUM = 36
TARGET_PROD = 360

def genotype_to_groups(gen: List[int]) -> Tuple[List[int], List[int]]:
    groupA = [CARDS[i] for i, b in enumerate(gen) if int(b) == 1]
    groupB = [CARDS[i] for i, b in enumerate(gen) if int(b) == 0]
    return groupA, groupB

def product_of(seq: List[int]) -> int:
    p = 1
    for v in seq:
        p *= int(v)
    return p if seq else 1

# ----- Sphere mapping and fitness -----
def encode_errors_to_vector(gen: List[int], scale_sum: float = 10.0, eps: float = 1e-9):
    groupA, groupB = genotype_to_groups(gen)
    sumA = sum(groupA)
    prodB = product_of(groupB)
    x1 = (sumA - TARGET_SUM) / float(scale_sum)
    x2 = math.log(prodB + eps) - math.log(TARGET_PROD)
    return (x1, x2), sumA, prodB

def sphere_value_from_genotype(gen: List[int], scale_sum: float = 10.0) -> float:
    (x1, x2), _, _ = encode_errors_to_vector(gen, scale_sum=scale_sum)
    return float(x1*x1 + x2*x2)

def score_from_sphere(fval: float) -> float:
    # Map minimization to maximization for selection (higher is better)
    return 1.0 / (1.0 + fval)

# ----- GA utilities -----
def random_genotype(rng: np.random.Generator) -> List[int]:
    return [int(rng.integers(0,2)) for _ in range(10)]

def tournament_select(pop: List[List[int]], scores: List[float], rng: np.random.Generator,
    tsz=3):
    # return a copy of selected genotype

```

```

idxs = rng.integers(0, len(pop), size=tsize)
best = idxs[0]
for i in idxs:
    if scores[i] > scores[best]:
        best = i
return pop[best].copy()

```

```

def single_point_crossover(p1: List[int], p2: List[int], rng: np.random.Generator):
    n = len(p1)
    if n <= 1 or rng.random() < 1e-12:
        return p1.copy(), p2.copy()
    cx = rng.integers(1, n) # crossover point
    c1 = p1[:cx] + p2[cx:]
    c2 = p2[:cx] + p1[cx:]
    return c1, c2

```

```

def bit_flip_mutation(gen: List[int], mut_rate: float, rng: np.random.Generator):
    out = gen.copy()
    for i in range(len(out)):
        if rng.random() < mut_rate:
            out[i] = 1 - out[i]
    return out

```

----- Generational GA -----

```

def run_generational_ga(rng: np.random.Generator,
    pop_size: int = 100,
    generations: int = 200,
    mut_rate: float = 0.01,
    scale_sum: float = 10.0,
    elitism: int = 1) -> Tuple[float, List[int]]:
    # initialize
    pop = [random_genotype(rng) for _ in range(pop_size)]
    fvals = [sphere_value_from_genotype(g, scale_sum) for g in pop]
    best_idx = int(np.argmin(fvals))
    best_gen = pop[best_idx].copy()
    best_f = fvals[best_idx]

    for gen in range(generations):
        scores = [score_from_sphere(f) for f in fvals]
        new_pop = []
        # elitism: carry best individuals

```

```

sorted_idx = sorted(range(pop_size), key=lambda i: fvals[i])
for i in range(elitism):
    new_pop.append(pop[sorted_idx[i]].copy())
# generate rest
while len(new_pop) < pop_size:
    parent1 = tournament_select(pop, scores, rng)
    parent2 = tournament_select(pop, scores, rng)
    c1, c2 = single_point_crossover(parent1, parent2, rng)
    c1 = bit_flip_mutation(c1, mut_rate, rng)
    c2 = bit_flip_mutation(c2, mut_rate, rng)
    new_pop.append(c1)
    if len(new_pop) < pop_size:
        new_pop.append(c2)
pop = new_pop
fvals = [sphere_value_from_genotype(g, scale_sum) for g in pop]
cur_best_idx = int(np.argmin(fvals))
if fvals[cur_best_idx] < best_f:
    best_f = fvals[cur_best_idx]
    best_gen = pop[cur_best_idx].copy()
return best_f, best_gen

```

----- Steady-State GA -----

```

def run_steady_state_ga(rng: np.random.Generator,
    pop_size: int = 100,
    iterations: int = 1000,
    mut_rate: float = 0.01,
    scale_sum: float = 10.0,
    replace_count: int = 2) -> Tuple[float, List[int]]:
    # initialize
    pop = [random_genotype(rng) for _ in range(pop_size)]
    fvals = [sphere_value_from_genotype(g, scale_sum) for g in pop]
    best_idx = int(np.argmin(fvals))
    best_gen = pop[best_idx].copy()
    best_f = fvals[best_idx]

    for it in range(iterations):
        # create a small number of offspring
        scores = [score_from_sphere(f) for f in fvals]
        parent1 = tournament_select(pop, scores, rng)
        parent2 = tournament_select(pop, scores, rng)
        c1, c2 = single_point_crossover(parent1, parent2, rng)

```

```

c1 = bit_flip_mutation(c1, mut_rate, rng)
c2 = bit_flip_mutation(c2, mut_rate, rng)
off = [c1, c2][:replace_count]
off_f = [sphere_value_from_genotype(o, scale_sum) for o in off]
# replace worst individuals if offspring better
for o_gen, o_f in zip(off, off_f):
    worst_idx = int(np.argmax(fvals))
    if o_f < fvals[worst_idx]:
        pop[worst_idx] = o_gen
        fvals[worst_idx] = o_f
    if o_f < best_f:
        best_f = o_f
        best_gen = o_gen.copy()
return best_f, best_gen

# ----- Experiment runner -----
def run_experiments(repeats: int = 50,
                    pop_size: int = 100,
                    gen_generations: int = 200,
                    ss_iterations: int = 1000,
                    mut_rate: float = 0.01,
                    scale_sum: float = 10.0,
                    out_csv: str = "ga_card_game_results.csv"):
    gen_results = []
    ss_results = []

    for run in range(repeats):
        seed = 1000 + run
        rng = np.random.default_rng(seed)
        bf, bg = run_generational_ga(rng, pop_size=pop_size, generations=gen_generations,
                                     mut_rate=mut_rate, scale_sum=scale_sum, elitism=1)
        gen_results.append((bf, bg))
        # steady-state uses different seed for independence
        rng2 = np.random.default_rng(seed + 50000)
        bf2, bg2 = run_steady_state_ga(rng2, pop_size=pop_size, iterations=ss_iterations,
                                       mut_rate=mut_rate, scale_sum=scale_sum, replace_count=2)
        ss_results.append((bf2, bg2))
        print(f"Run {run+1}/{repeats}: Generational best={bf:.6e}, SteadyState
best={bf2:.6e}")

# extract numeric arrays

```

```

gen_vals = np.array([b for b,_ in gen_results], dtype=float)
ss_vals = np.array([b for b,_ in ss_results], dtype=float)

def stats(arr):
    return {
        'mean': float(np.mean(arr)),
        'median': float(np.median(arr)),
        'var': float(np.var(arr, ddof=0)),
        'std': float(np.std(arr, ddof=0)),
        'min': float(np.min(arr)),
        'max': float(np.max(arr))
    }

gen_stats = stats(gen_vals)
ss_stats = stats(ss_vals)

print("\n--- Summary (Sphere fitness, lower is better) ---")
print("Generational GA:", gen_stats)
print("Steady-State GA:", ss_stats)

# Save per-run results and genotypes to CSV for inspection
with open(out_csv, "w", newline="") as f:
    writer = csv.writer(f)
    writer.writerow(["mode","run","best_sphere","genotype","sumA","prodB"])
    for i, (b,g) in enumerate(gen_results):
        _, sumA, prodB = encode_errors_to_vector(g)
        writer.writerow(["generational", i+1, b, "".join(map(str,g)), sumA, prodB])
    for i, (b,g) in enumerate(ss_results):
        _, sumA, prodB = encode_errors_to_vector(g)
        writer.writerow(["steady_state", i+1, b, "".join(map(str,g)), sumA, prodB])

return gen_stats, ss_stats, gen_results, ss_results

# ----- Main -----
if __name__ == "__main__":
    # Parameters (changeable)
    REPEATS = 50
    POP_SIZE = 100
    GENERATIONAL_GENS = 200    # total generations for Generational GA
    STEADY_STATE_ITERS = 1000  # more iterations as requested for Steady-State GA
    MUT_RATE = 0.01

```

SCALE_SUM = 10.0

```
run_experiments(repeats=REPEATS,  
                pop_size=POP_SIZE,  
                gen_generations=GENERATIONAL_GENS,  
                ss_iterations=STEADY_STATE_ITERS,  
                mut_rate=MUT_RATE,  
                scale_sum=SCALE_SUM,  
                out_csv="ga_card_game_results.csv")
```